

```

// Test bench for the binary multiplier
module t_Sequential_Binary_Multiplier;
    parameter    dp_width = 5;                // Set to width of datapath
    wire         [2*dp_width -1: 0] Product;    // Output from multiplier
    wire         Ready;
    reg          [dp_width -1: 0]  Multiplicand, Multiplier; // Inputs to multiplier
    reg          Start, clock, reset_b;
// Instantiate multiplier
Sequential_Binary_Multiplier M0 (Product, Ready, Multiplicand, Multiplier, Start,
clock, reset_b);
    // Generate stimulus waveforms
initial #200 $finish;
initial
    begin
        Start = 0;
        reset_b = 0;
        #2 Start = 1; reset_b = 1;
        Multiplicand = 5'b10111;    Multiplier = 5'b10011;
        #10 Start = 0;
    end
initial
    begin
        clock = 0;
        repeat (26) #5 clock = ~clock;
    end
// Display results and compare with Table 8.5
always @ (posedge clock)
    $strobe ("C=%b A=%b Q=%b P=%b time=%0d",M0.C,M0.A,M0.Q,M0.P, $time);
endmodule

/*
Simulation log:
C=0 A=00000    Q=10011    P=101 time=5
C=0 A=10111    Q=10011    P=100 time=15
C=0 A=01011    Q=11001    P=100 time=25
C=1 A=00010    Q=11001    P=011 time=35
C=0 A=10001    Q=01100    P=011 time=45
C=0 A=10001    Q=01100    P=010 time=55
C=0 A=01000    Q=10110    P=010 time=65
C=0 A=01000    Q=10110    P=001 time=75
C=0 A=00100    Q=01011    P=001 time=85
C=0 A=11011    Q=01011    P=000 time=95
C=0 A=01101    Q=10101    P=000 time=105
C=0 A=01101    Q=10101    P=000 time=115
C=0 A=01101    Q=10101    P=000 time=125
*/

/* Test bench for exhaustive simulation
module t_Sequential_Binary_Multiplier;
    parameter    dp_width = 5;                // Width of datapath
    wire         [2 * dp_width -1: 0] Product;
    wire         Ready;
    reg          [dp_width -1: 0]  Multiplicand, Multiplier;

```

```

reg                                Start, clock, reset_b;
    Sequential_Binary_Multiplier M0 (Product, Ready, Multiplicand, Multiplier, Start,
        clock, reset_b);
initial #1030000 $finish;
initial begin clock = 0; #5 forever #5 clock = ~clock; end
initial fork
    reset_b = 1;
    #2 reset_b = 0;
    #3 reset_b = 1;
join
initial begin #5 Start = 1; end
initial begin
    #5 Multiplicand = 0;
    Multiplier = 0;
    repeat (32) #10 begin Multiplier = Multiplier + 1;
        repeat (32) @ (posedge M0.Ready) #5 Multiplicand = Multiplicand + 1;
    end
end
endmodule
*/

```
